

УНИВЕРСИТЕТ ИТМО

Факультет прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Дисциплина «Бекэнд-разработка»



ОТЧЕТ по

лабораторной работе 1

“Переход к микросервисной архитектуре”

Выполнил:

Таипов Т.А.

Группа:

К3441

Преподаватель:

Добряков Д.И.

Санкт-Петербург

2026 г.

Содержание

ВВЕДЕНИЕ	2
1 Задача	2
2 Ход работы	2
2.1 Краткое описание предметной области	2
2.2 Цели перехода к микросервисам	2
2.3 Выделение самостоятельных модулей	2
2.4 Маппинг модулей на сервисы	3
2.5 Разделение API на микросервисы	3
2.6 Границы ответственности и контракты API	3
2.7 Настройка сетевого взаимодействия	4
2.8 Схема развертывания и контейнеризация	4
2.9 Модель данных и изоляция сервисов	5
2.10 Авторизация и безопасность	5
2.11 Пользовательские сценарии	5
2.12 Проверка работоспособности	6
2.13 Набор тестовых сценариев	6
2.14 Обработка ошибок и отказоустойчивость	6
2.15 Проблемы и решения в ходе работы	7
2.16 Оценка достигнутых результатов	7
2.17 Итоговая структура решения	7
3 Вывод	7

ВВЕДЕНИЕ

В рамках лабораторной работы выполнен переход исходного backend-приложения к микросервисной архитектуре. Работа проведена на проекте `jobboard-microservices`.

1 ЗАДАЧА

Текст задачи на лабораторную/практическую работу:

- выделить самостоятельные модули в приложении;
- провести разделение API на микросервисы (минимум 3 сервиса);
- настроить сетевое взаимодействие между микросервисами.

2 ХОД РАБОТЫ

2.1 Краткое описание предметной области

Проект представляет собой сервис для поиска работы и взаимодействия работодателей с кандидатами. В системе выделяются следующие ключевые сущности: пользователи (кандидаты/работодатели), профили кандидатов, компании, вакансии, отклики и сообщения по откликам. Для поддержки масштабируемости и независимого развития функциональных частей приложение переведено на микросервисную архитектуру.

2.2 Цели перехода к микросервисам

Переход к микросервисам выполнен для решения следующих задач:

- **Снижение связности** между доменными областями (пользователи, вакансии, отклики);
- **Независимое масштабирование** сервисов с различной нагрузкой;
- **Ускорение разработки** за счет четких границ ответственности;
- **Гибкость развертывания** и упрощение сопровождения.

2.3 Выделение самостоятельных модулей

В приложении выделены следующие доменные модули:

- **Управление пользователями** — регистрация, вход, роли, профили кандидатов;
- **Управление вакансиями** — вакансии, компании, индустрии, локации;
- **Отклики и коммуникации** — отклики на вакансии и сообщения по откликам.

Такое разбиение уменьшает связность между частями системы и позволяет независимо развивать каждый функциональный блок.

2.4 Маппинг модулей на сервисы

Соответствие доменных модулей и микросервисов приведено в таблице 1.

Модуль	Микросервис и функции
Управление пользователями	User Service: регистрация, авторизация, роли, профили кандидатов
Управление вакансиями	Job Service: компании, вакансии, индустрии, локации
Отклики и коммуникации	Application Service: отклики на вакансии и сообщения
Маршрутизация API	API Gateway: единая точка входа, проверка JWT, проксирование

Таблица 1 – Соответствие модулей и сервисов

2.5 Разделение API на микросервисы

На основе выделенных модулей API разделено на три основных микросервиса и один инфраструктурный сервис-шлюз:

- **User Service (:4001)** — маршруты /api/users, /api/candidate-profiles;
- **Job Service (:4002)** — маршруты /api/jobs, /api/companies, /api/industries, /api/locations;
- **Application Service (:4003)** — маршруты /api/applications, /api/messages;
- **API Gateway (:4000)** — единая точка входа, проксирование и проверка JWT для защищенных маршрутов.

Таким образом, требование “минимум 3 микросервиса” выполнено.

2.6 Границы ответственности и контракты API

Для каждого сервиса определен собственный набор REST-эндпоинтов. Это позволяет избежать пересечения ответственности и упрощает поддержку контрактов. Сводка приведена ниже:

- **User Service:** GET /users, POST /users/register, POST /users/login, GET /candidate-profiles;
- **Job Service:** GET /jobs, POST /jobs, GET /companies, POST /companies;
- **Application Service:** GET /applications, POST /applications, GET /messages, POST /messages.

Для внешних клиентов эти маршруты доступны только через API Gateway, что позволяет централизовать контроль доступа.

2.7 Настройка сетевого взаимодействия

Сетевое взаимодействие настроено через docker-compose:

- все сервисы подключены к общей сети jobboard-network;
- развёрнута общая БД PostgreSQL (postgres, порт 5432 внутри сети, 5435 на хосте);
- каждый микросервис получает доступ к БД через переменную DATABASE_URL;
- межсервисное взаимодействие выполняется по внутренним DNS-именам контейнеров.

Основные связи между сервисами:

- клиент обращается только к API Gateway;
- API Gateway маршрутизирует запросы в соответствующий микросервис;
- Application Service выполняет HTTP-запрос к Job Service для проверки вакансии перед созданием отклика;
- все сервисы взаимодействуют с PostgreSQL для хранения и чтения данных.

2.8 Схема развертывания и контейнеризация

Все сервисы запускаются в Docker-контейнерах. Пример ключевой части файла docker-compose.yml:

```
services:
  postgres:
    image: postgres:latest
    ports: ["5435:5432"]
    networks: [jobboard-network]

  user-service:
    build: ./user-service
    ports: ["4001:4001"]
    depends_on: [postgres]

  job-service:
    build: ./job-service
    ports: ["4002:4002"]
```

```
depends_on: [postgres, user-service]
```

```
application-service:
```

```
  build: ./application-service
```

```
  ports: ["4003:4003"]
```

```
  depends_on: [postgres, user-service, job-service]
```

```
api-gateway:
```

```
  build: ./api-gateway
```

```
  ports: ["4000:4000"]
```

```
  depends_on: [user-service, job-service, application-service]
```

2.9 Модель данных и изоляция сервисов

Каждый сервис отвечает за свой набор сущностей. На уровне БД соблюдается логическая изоляция доменов:

- **User Service**: пользователи и профили кандидатов;
- **Job Service**: компании, вакансии, справочники индустрий и локаций;
- **Application Service**: отклики и сообщения.

Хотя используется единая БД PostgreSQL, ответственность за таблицы разделена и отражена в коде через отдельные модели TypeORM внутри каждого сервиса.

2.10 Авторизация и безопасность

Проверка доступа реализована через JWT:

- токен выдается в `User Service` при успешном логине;
- `API Gateway` валидирует JWT на защищенных маршрутах;
- `Job Service` дополнительно проверяет роль пользователя (например, `employer`) при создании вакансий и компаний.

Такой подход обеспечивает централизованный контроль доступа и разграничение прав на уровне бизнес-операций.

2.11 Пользовательские сценарии

Основные пользовательские сценарии, которые поддерживает система:

1. **Регистрация и вход**: пользователь создает учетную запись, получает JWT-токен и использует его для доступа к защищенным операциям.

2. **Публикация вакансии:** работодатель создает компанию и размещает вакансию через Job Service.
3. **Отклик на вакансию:** кандидат отправляет отклик через Application Service, который проверяет статус вакансии в Job Service.

2.12 Проверка работоспособности

Для проверки корректности развертывания были выполнены следующие шаги:

- запуск всех контейнеров через `docker-compose up`;
- проверка доступности API Gateway на `http://localhost:4000`;
- тестовые запросы к сервисам: создание пользователя, компании, вакансии, отклика.

2.13 Набор тестовых сценариев

Для ручной проверки корректности взаимодействия сервисов были выполнены следующие действия:

1. Регистрация пользователя-кандидата и получение JWT-токена.
2. Регистрация пользователя-работодателя и получение JWT-токена.
3. Создание компании работодателем.
4. Публикация вакансии работодателем.
5. Запрос списка вакансий кандидатом.
6. Отправка отклика кандидатом на опубликованную вакансию.
7. Получение списка откликов и сообщений по отклику.

Данные шаги покрывают основные пользовательские роли и проверяют корректность маршрутизации запросов через API Gateway.

2.14 Обработка ошибок и отказоустойчивость

В рамках работы особое внимание уделено корректной обработке ошибок:

- при отсутствии данных сервисы возвращают `404 Not Found`;
- при нарушении прав доступа возвращается `403 Forbidden`;
- при отсутствии/некорректном JWT — `401 Unauthorized`;
- при ошибках валидации входных данных — `400 Bad Request`.

Это обеспечивает предсказуемое поведение API и упрощает интеграцию клиентской части.

2.15 Проблемы и решения в ходе работы

В процессе миграции к микросервисной архитектуре были учтены следующие моменты:

- необходимость централизованной аутентификации решена через JWT и API Gateway;
- предотвращение жесткой связанности сервисов обеспечено разделением доменных областей;
- межсервисные вызовы из Application Service в Job Service используются только для проверки вакансий.

2.16 Оценка достигнутых результатов

В результате перехода:

- выделены четкие границы ответственности между командами и компонентами;
- упрощено масштабирование сервисов с различной нагрузкой;
- подготовлена основа для дальнейшего развития (логирование, мониторинг, брокер сообщений).

2.17 Итоговая структура решения

Client -> API Gateway (:4000)

-> User Service (:4001)

-> Job Service (:4002)

-> Application Service (:4003)

Application Service -> Job Service (проверка вакансии)

User/Job/Application Services -> PostgreSQL

3 ВЫВОД

В ходе лабораторной работы выполнено требуемое преобразование backend-приложения в микросервисную архитектуру: выделены самостоятельные модули, API разделено на 3 доменных микросервиса, настроено сетевое взаимодействие между сервисами через API Gateway, Docker-сеть и HTTP-вызовы. Полученная архитектура обеспечивает лучшую модульность, масштабируемость и удобство дальнейшей разработки.